

전기차 충전기 Predictive Maintenance

Break-Fix → Predict-Fix

1. 문제 정의
2. PdM 시스템 개요
3. 시스템 개발 방법
4. 개발 성과
5. Next step

01 | 문제 정의 — 충전기 고장



대한민국은 전 세계에서 충전기 1대당 전기차 수가 가장 적은, 충전 인프라 1위 국가이나, 급속충전 이용 고객 대다수가 충전기 고장 문제로 충전 실패를 경험

차종비	대한민국	OECD 평균
완속 충전기 1기당 전 기차 수	약 2.1대	약 12~15대
급속 충전기 1기당 전 기차 수	약 16.7대	약 60~70대
전체 합계	약 1.85대	약 10대

vs.

73%

급속충전 10회 중
1회 이상 실패 경험

컨슈머인사이트 2025

47%

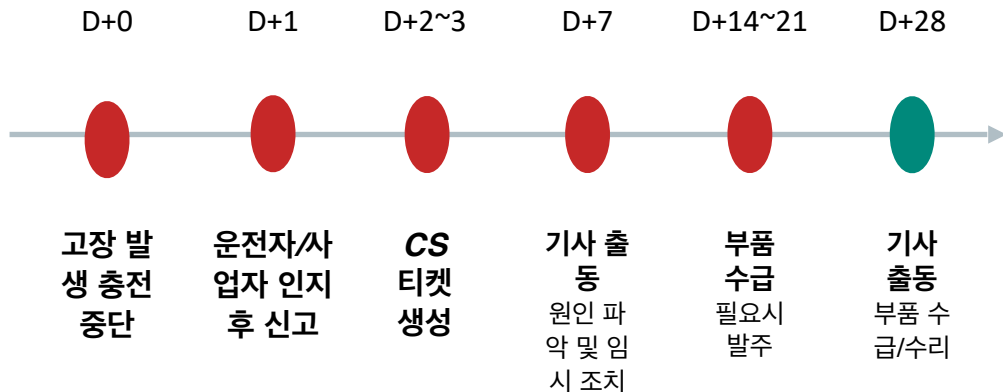
충전 실패 주원인은
충전기 고장

소프트베리 2023

01 | 문제 정의 — 사후 수리 방식에 기인한 악순환



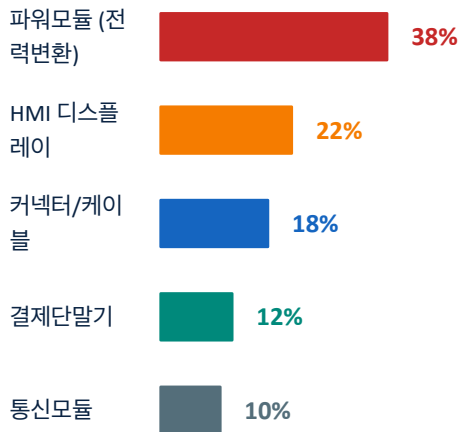
‘사후 수리’ 방식 대응에 따라 고장 발생부터 최종 수리까지 1주~1개월 소요되어 고객, 사업자, 제조사 모두가 손해를 보는 소모적 구조 고착화



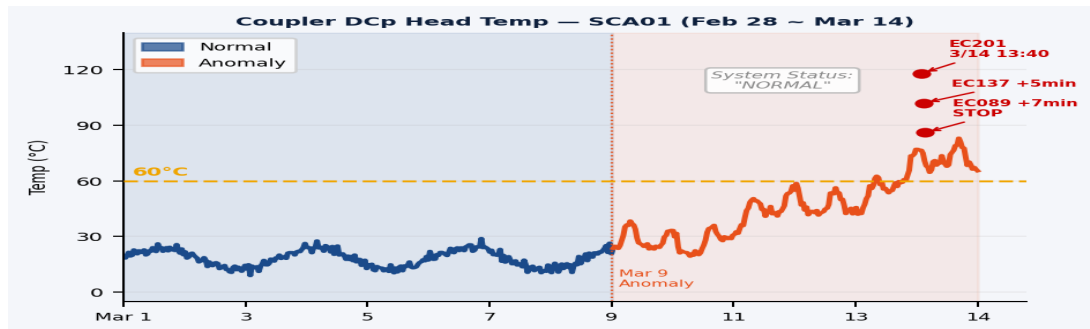
02 | AI Predictive Maintenance(이하 'PdM') 시스템 – 탐지 메커니즘 가설

부품 열화(劣化) 메커니즘상 고장의 전조 증상인 열 스트레스와 전기적 특성 변화를 찾아내면 고장 예측할 수 있을 것이다!

충전기 부품별 고장 비중



AI PdM 사전 탐지 메커니즘 가설



← AI PdM이 이 구간에서 이상 신호를 포착 →

02 | AI PdM 시스템 - 구조 & Output

AI PdM 전체 구조

▶ 데이터 수집

OCPP 로그 · 센서(온도/전류) · 정비 이력

▶ 피처 엔지니어링

온도 트렌드 · 효율 저하율 · 에러 빈도 · 열저항

▶ 이상 탐지

Isolation Forest

라벨 불필요
즉시 적용 가능

XGBoost / LSTM

라벨 00건 후 적용
고장 확률 예측

▶ 이상 원인 설명

"왜 이상인가" 근거 자동 생성 (SHAP)

▶ 진단 보고서 작성

매뉴얼 검색 → 진단 보고서 자동 작성 (RAG + LLM)

▶ 액션

현장 기사 알람 + 부품 발주 자동 생성



예상 Output

충전기	실제 고장일	AI 감지일	탐지	주요 신호
-----	--------	--------	----	-------

A-047	2024-03-15	2024-03-08	✓	7일 전 temp_per_kw +23% (열저항 증가 신호)
-------	------------	------------	---	--------------------------------------

B-112	2024-05-22	2024-05-19	✓	3일 전 에러코드 반복 12회 (평균 1.3회의 9배)
-------	------------	------------	---	-----------------------------------

D-201	2024-09-01	2024-08-24	✓	8일 전 efficiency_drop_7d -4.2%p/주 (효율 하락 신호)
-------	------------	------------	---	--

02 | AI PdM 시스템 – 기대 효과

AS-IS | Break-Fix (현재)

- ✗ 고장 발생 후 신고 접수
- ✗ 기사 현장 출동 (1~2일 소요)
- ✗ 부품 수급 대기 (최대 수 주)
- ✗ 전체 수리 1주 ~ 1개월
- ✗ 충전서비스회사(CPO) 매출 손실 + 고객 불만
- ✗ 제조사 브랜드 훼손



TO-BE | Predict-Fix (목표)

- ✓ AI가 N일 전 이상 징후 포착
- ✓ 부품 선조달 + 예방 출동 → 수리 3~7일
- ✓ CPO 가동률 97~99% 달성
- ✓ 고객 불만 Zero
- ✓ 제조사 AS 비용 부담 경감 및 평판 개선

제조사 수익모델 전환/확장

하드웨어 판매

→ HW + PdM SaaS 구독 번들

보증 기간 내 무상 수리

→ Uptime SLA 계약
(가동률 보장 대가 수익)

* SLA: Service Level Agreement

03 | AI PdM 시스템 개발 — 개발 방법과 여정

개발 방법 — 비개발자의 Vibe Coding

Prior coding experience: Zero

| 개발 기간: 2 개월 | 혼자



Claude + Cursor

자연어 명령 → 코드 자동 생성
전체 PdM 시스템 8개 모듈 구축

프롬프트 예시: Cursor 에러를 그대로 Claude에 붙여넣고
"이 에러를 어떻게 고치면 되나요?" 질문



Python ML

Isolation Forest + XGBoost + SHAP
비지도·지도학습 앙상블

개발 여정 — 피벗과 시행착오

① 고장 1위(38%) 파워모듈 PdM의 MVP 개발·검증 후 확대 계획
→ 데이터 부재로 케이블 커넥터(3위)로 타겟 전환

② (데이터 수령 전) NASA Public 데이터로 알고리즘 사전 검증
1차 탐지율 15.8% 실패 → 디버깅 → 66.7% 달성, 신뢰성 확인

* NASA가 2008년에 공개한 이 데이터셋은 PdM 연구의 표준 벤치마크로 쓰임. 항공기 엔진 100대를 신품 상태에서 완전 고장까지 돌리며 기록한 20,631행 데이터. 열화메커니즘이 충전기 파워모듈과 유사

③ 1대 1개월 데이터로 가능성 확인 후 → 46대 (정상+고장), 2개월 데이터로 스케일업

46 대

실제 충전기
정상 16 + 고장 30

774,048 행

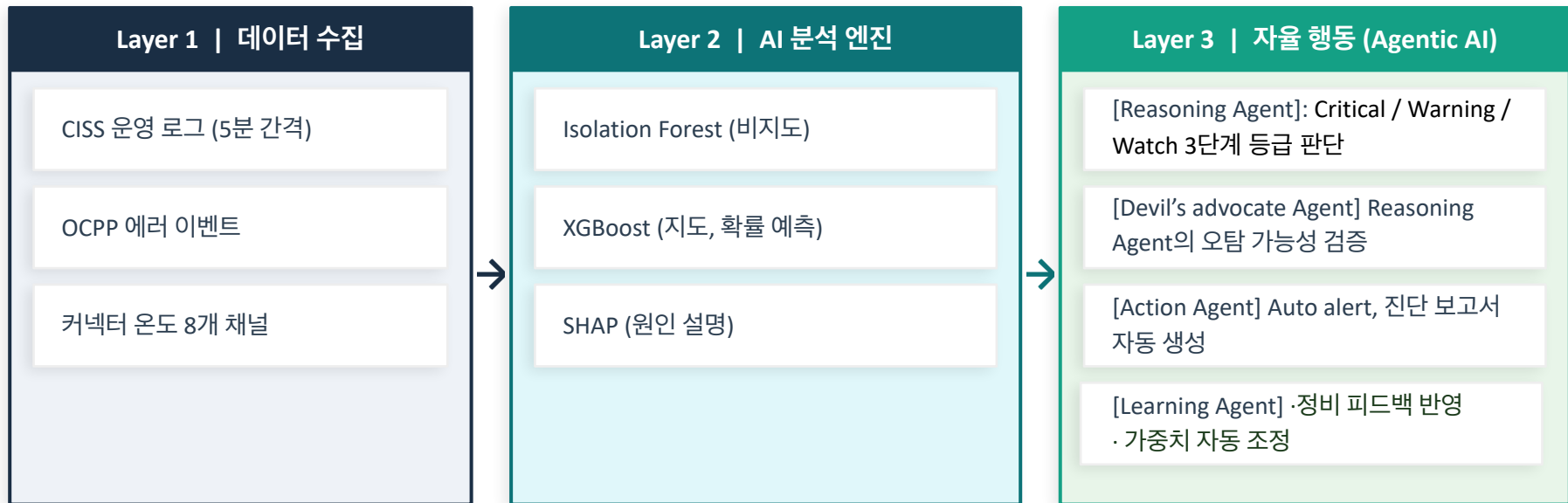
운영 로그
2개월 5분 간격

비정형

고장사례 DB,
에러코드, 정비매뉴얼,

④ 중간발표 피드백에 따라 Agentic AI도 추가 구현

03 | AI PdM 시스템 개발 — 시스템 구조



* *Reasoning Agent*: 모델이 출력한 이상 스코어·고장 확률·에러코드를 입력받아 *CRITICAL / WARNING / WATCH* 3단계 등급으로 변환하는 판단 엔진.

* 진단보고서: EC딕셔너리·정비 매뉴얼·고장 사례 DB를 실시간으로 검색하여 현장 기사가 즉시 행동할 수 있는 구체적 지침을 0.3초 내 자동 작성.

* *Learning Agent*: 현장 기사의 정비 완료 피드백을 바탕으로 신호별 가중치를 자동 조정하는 자기 개선 메커니즘.

Isolation Forest

이상 탐지 (비지도 학습)

데이터 수집 즉시 적용

새로운 패턴도 탐지

이상 스코어 < -0.25

* *Isolation Forest* 이상 스코어: 데이터가 정상 패턴에서 얼마나 벗어났는지를 표현. 경험적으로 $-0.5 \sim +0.5$ 에 분포하며 클수록 정상. 본 시스템은 탐지율과 오경보의 조합이 가장 좋았던 -0.25 를 임계값으로 설정(-0.15 는 탐지율 개선되나 경보 남발)

XGBoost

고장 확률 예측 (지도 학습)

고장 사례 30건 학습

0~100% 확률 출력

prob $\geq 45\%$ → 알람

* *XGBoost* 고장 확률: 실제 고장 사례 30건을 학습하여 7일 내 고장으로 이어질 확률을 0~100%로 출력. 임계값을 45%로 설정한 것은 고장 미탐지 시 손실(\$2,400+)이 가파 경보 시 손실보다 크다는 비대칭적 비용 구조를 반영하기 위함

03 | AI PdM 시스템 개발 —설명 엔진 (SHAP)

❌ AI만으로는 부족하다

"고장 위험 84%"

어디가? 왜?

→ 기사: "또 오작동이겠지"



✅ SHAP = AI의 판결문 — 판단 근거 정량적 제시

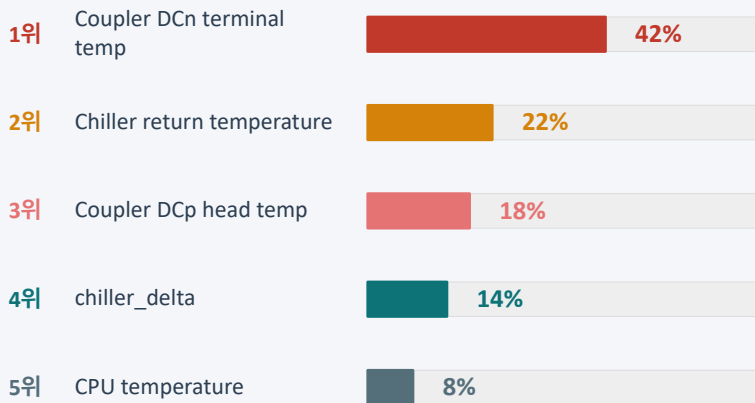
"고장 84%"

Coupler DCn 42%

+ EC 반복 31% + 냉각 15%"



"온도를 보라" 라고 가르친게 아닌데 AI가 스스로 피쳐 중요도 발견"



"고장 연쇄 메커니즘까지도 발견"

- 1 Chiller 냉각 효율 저하**
chiller_delta 증가 → 케이블 냉각 악화
- 2 Coupler 온도 60°C 돌파**
AI 경보 발령 / 공식 경보(90°C)보다 30°C 먼저 → 19일 확보
- 3 PP 전압 에러 (EC74) 발생**
DC 핀 접촉 불량 신호 → 확신도 90% → AUTO_ALERT
- 4 완전 고장 (EC49 반복)**
EC49→EC74 연쇄 → 충전기 정지 ← Break-Fix 인지 시점

03 | AI PdM 시스템 개발 — Agentic AI (스스로 판단, 검증, 행동, 학습)



[실시간 & 사람 개입 없이 판단, 검증, 행동, 학습]

RRG01 → EC 감지 → 확신도 90% → Devil's Advocate: PROCEED (오탐 이력 없음, 데이터 정상) → AUTO_ALERT 자동 발령

04 | AI PdM 개발 성과 - FDC02 사례

● 2월 28일

AI 경보 발령

온도는 아직 정상권이었다.
그러나 AI는 커넥터 열화 패턴의 변화를 읽었다.

● 3월 6일

이상 탐지 — 온도 점진적 상승 시작

Isolation Forest 이상 스코어 임계값 초과
시스템 상태: 여전히 '정상'.

● 3월 13~19일

가속 열화 — 60°C 반복 초과

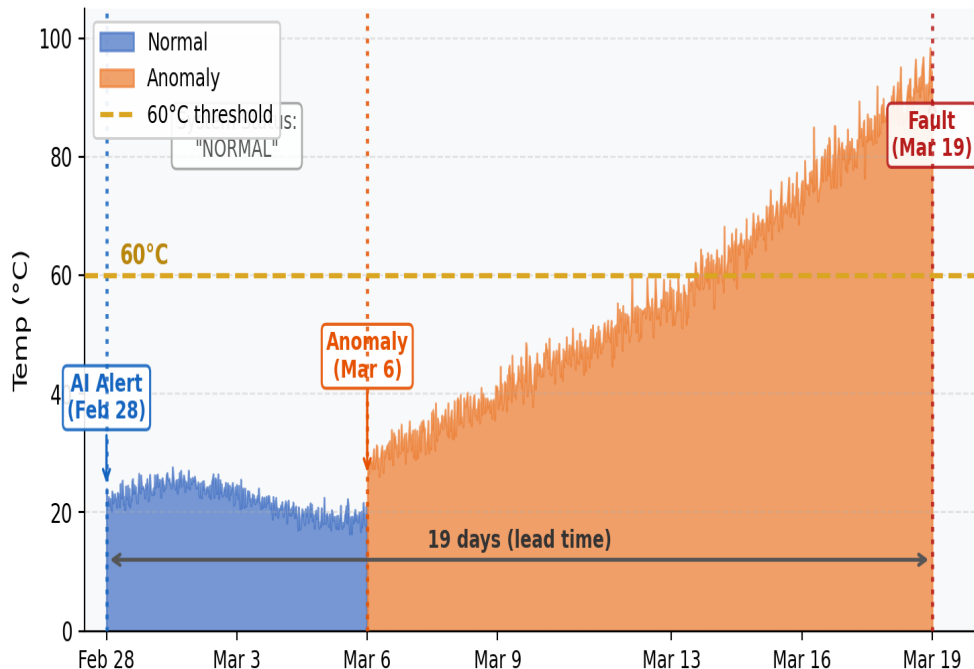
커넥터 온도 60°C 상시 초과, 지속 상승
XGBoost 고장 확률 45% 돌파 → AUTO_ALERT

🚫 3월 19일

충전기 정지 (기존 시스템 고장 확정)

커넥터 DC핀 열화 → 접촉 불량으로 정지
AI 경보 발령으로부터 19일 후

Coupler DCp Head Temp — FDC02 (Feb 28 ~ Mar 19, 19 days)



AI는 19일 전인 2월28일에 경보를 발령했다. Break-Fix 방식이었다면 3월 14일이 되어서야 알았을 것이다.

04 | AI PdM 개발 성과 - 종합

92.3%

탐지율

13건 중 12건 고장 전 경보

19일

평균 조기 경보

공식 EC 경보보다

평균 19일 먼저

3~51일

리드타임

반드시 대응할 시간이 있다

사이트별 실증 결과

사이트	고장일	AI 경보일	리드타임	의미
SCA01	2026-04-20	2026-02-28	51일	51일 전이면 부품 선발주 통해 대기 없이 수리 가능
FDC01	2026-03-10	2026-02-28	10일	출장 일정 미리 조율 가능
FDC02	2026-03-19	2026-02-28	19일	출장 일정 미리 조율 가능
RSA03	2026-03-03	2026-02-28	3일	최소 리드타임
RRG01	진행 중	2026-05-20	실시간	고장 발생 전 AUTO_ALERT 발령 중
FFG01	2026-04-21	—	—	운영 데이터 없음 → 있었다면 탐지율 100% 달성 가능(모델 한계 아님)

04 | AI PdM 개발 성과 – 진단보고서



EC 코드 디렉터리

126개 원인·조치·카테고리 완비



정비 매뉴얼

CCS 교체 26단계

토크 규격·냉각수 절차



고장 사례 DB

5건 구조화

유사도 검색으로 즉시 참조



실시간 알람 현황

현재 13대 ADVISORY 경보



PdM 진단 보고서 | RRG01 | AUTO_ALERT

추정 원인

EC74 (PP 전압 에러) 감지

→ DC핀 접촉 불량, 커넥터 열화

권장 조치

① LOTO 수행 (잔류전압 2분 대기)

② CCS 커넥터 핀 산화·파손 점검

③ DC 토크 $22 \pm 10\%$ N·m 재조임

④ 냉각수 레벨 확인 및 보충

출동 준비

부품: E41315-S00004 (CCS cable)

소요: 약 1시간 | 냉각수 보충 필요

유사 사례

[CCR02 2025-09-12] EC49 동일 패턴

→ 커넥터 교체로 해결 (3시간)

현장 기사는 이 보고서로 부품을 챙기고, 무엇을 볼 지 미리 알고 출동

05 | Next step

현재 완성

이미 작동하고 있다

커넥터 PdM

탐지율 92.3% / 리드타임 19일

Agentic AI 5단계

Perception·Reasoning(1/2)·Action·Learning

EC 디셔너리 000개

원인·조치·카테고리 완비

RAG 진단 보고서

현장 기사용 자동 생성

Rule Engine + Learning

오진 시 자동 가중치 조정

6/19 도입
의사결정!!!

7/7
2차 워크샵 예정

1년차 로드맵

진짜 임팩트를 만든다

필드 파일럿 테스트

000대 이상 실제 서비스 적용

Signal Level 최적화

정비 완료 입력 → 모델 학습, 임계값 실전 데이터 보
정

Claude API 연동

자연어 진단 보고서 고도화

파워모듈 PdM 등으로 확대

Fan·온도 데이터 확보 후

리더의 문제의식/질문 + 데이터 + Grit
= *Breakthrough*
